

ros2can 講習

CAN バス対応 ROS 2 ⇔ マイコン通信パッケージ

立命館大学 ロボット技術研究会 RRST NHK プロジェクト

本日の内容

1

ros2can とは

何ができるのか・システム構成・専用基板

2

GUI を使ってみる

インストール、実機なしでの操作、安全機能

3

自作ノードから動かす

トピック、スロット割当、サンプルコード

4

応用

CubeMars AK・DJI ロボマス・ファームウェア生成

講習後にできること

- 実機なしで GUI を操作し、指令とセンサ値の流れを説明できる
- 自作 ROS 2 ノードからサーボを動かし、センサ値を読める
- 自分の基板用のファームウェアを生成・書き込みできる
- 安全な止め方 (E-STOP ・ゼロ指令) を知っている

01

ros2can とは

何ができるのか・どう繋がっているのか

ros2can とは

serial_bridge をベースに開発した後継パッケージ。CAN バスでマイコンを**デジチェーン接続**し、PyQt5 の GUI から直接操作できます。

1 CAN デジチェーン

PC とは USB1 本。残りのマイコンは電源 +CAN の 4 ピンケーブルで数珠つなぎ

2 GUI 搭載

Control / Monitor / Raw / Info の 4 タブで指令送信とセンサ確認

3 実機なしで試せる

仮想デバイスで画面やトピック連携をその場で確認

4 ロボマス・AK 対応

DJI ロボマス・CubeMars AK を専用モードで直接制御 (MIT モード等)

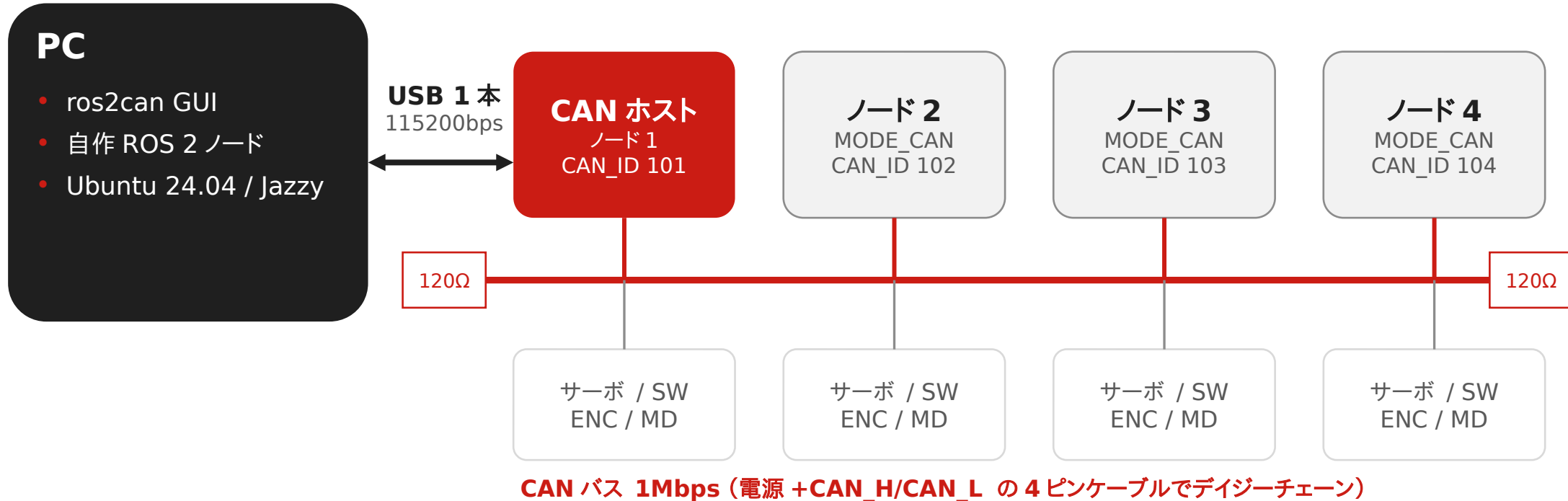
5 FW 生成・書き込み

マイコンの設定を GUI で入力し、生成から書き込みまで完結

6 serial_bridge 互換

serial_tx_[ID] / serial_rx_[ID] はそのまま。既存ノードを流用可能

システム構成



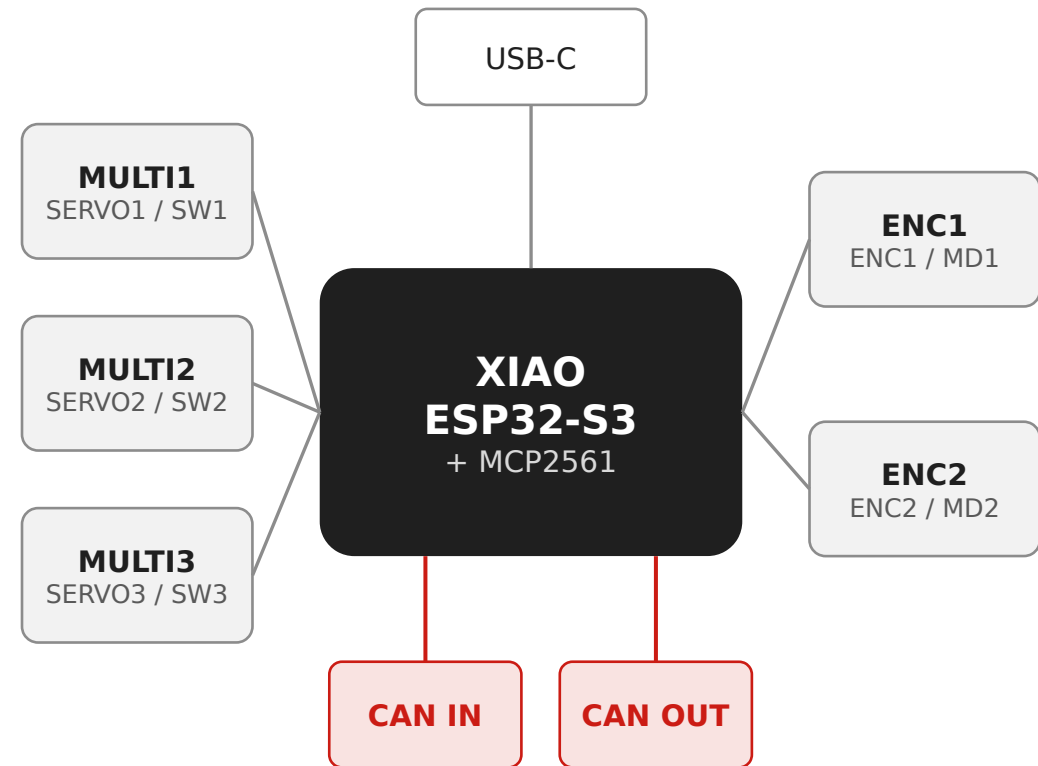
ポイント: PC⇄ホストは serial_bridge 互換の 24 × int16 フレーム。ホストがこの 24 スロットを各ノードへ 5 スロットずつ分配・集約します。
 ホスト自身もノード 1 として自分の I/O を直接処理します (CAN を経由しない)。

専用基板 (XIAO ESP32-S3)

ros2can 用に新規設計した小型基板。サーボ・エンコーダ・スイッチ・CAN コネクタを備え、電源 +CAN の 4 ピンケーブルでデバッグチェーンできます。

項目	内容
マイコン	Seeed XIAO ESP32-S3
CAN トランシーバ	MCP2561 (1Mbps)
MULTI1 ~ 3	スイッチ入力 or サーボ出力
ENC1 / ENC2	エンコーダ入力 or MD 出力 (PWM+DIR)
接続	USB-C (ホスト時) / CAN 4 ピン ×2

ピン共有に注意: どちらの用途で使うかはファームウェアの config.hpp (MULTIn / ENCN_MD) でコンパイル時に固定します。実配線と設定が一致しないと動きません。



ファームウェアの MODE

MODE	役割	ros2can での扱い
MODE_CAN_HOST	PC と USB 接続し、配下のノードを束ねる CAN ホスト(自身もノード 1)	CAN Host プロファイル(既定)
MODE_CAN	ホスト配下の汎用 IO ノード(PC とは直接繋がらない)	ホスト側プロファイルに内包
MODE_ROBOMAS	DJI ロボマス最大 4 台を直接駆動する独立デバイス	robomas_driver
MODE_CUBEMARS	CubeMars AK シリーズ最大 4 台を直接駆動する独立デバイス	cubemars_ak_driver
MODE_IO	CAN を使わない、IO のみのスタンドアロン動作	汎用 Raw 等
MODE_CAN_MONITOR	生 CAN フレームをシリアル出力するだけの診断用	ツールバー「CAN モニター…」
MODE_DEBUG	開発・デバッグ用	—

config.hpp で MODE を 1 つだけ定義します(複数定義・未定義はビルドエラー)。

02

GUI を使ってみる

インストールから、実機なしでの操作、安全な止め方まで

インストールと起動

1

依存パッケージを入れる

PyQt5 と pyserial

2

ポート権限を付与(初回のみ)

dialout グループに追加 → 再ログイン

3

ビルドして起動

ros2 run でも launch でも OK

4

自動でマイコンを検出

ホスト基板を USB に挿すだけで一覧に出る

1. 依存パッケージ

```
sudo apt install python3-pyqt5 python3-serial
```

2. ポート権限(初回のみ・要再ログイン)

```
sudo usermod -aG dialout $USER
```

3. ビルド

```
cd ~/ros2_ws
```

```
colcon build --packages-select ros2can
```

```
source install/setup.bash
```

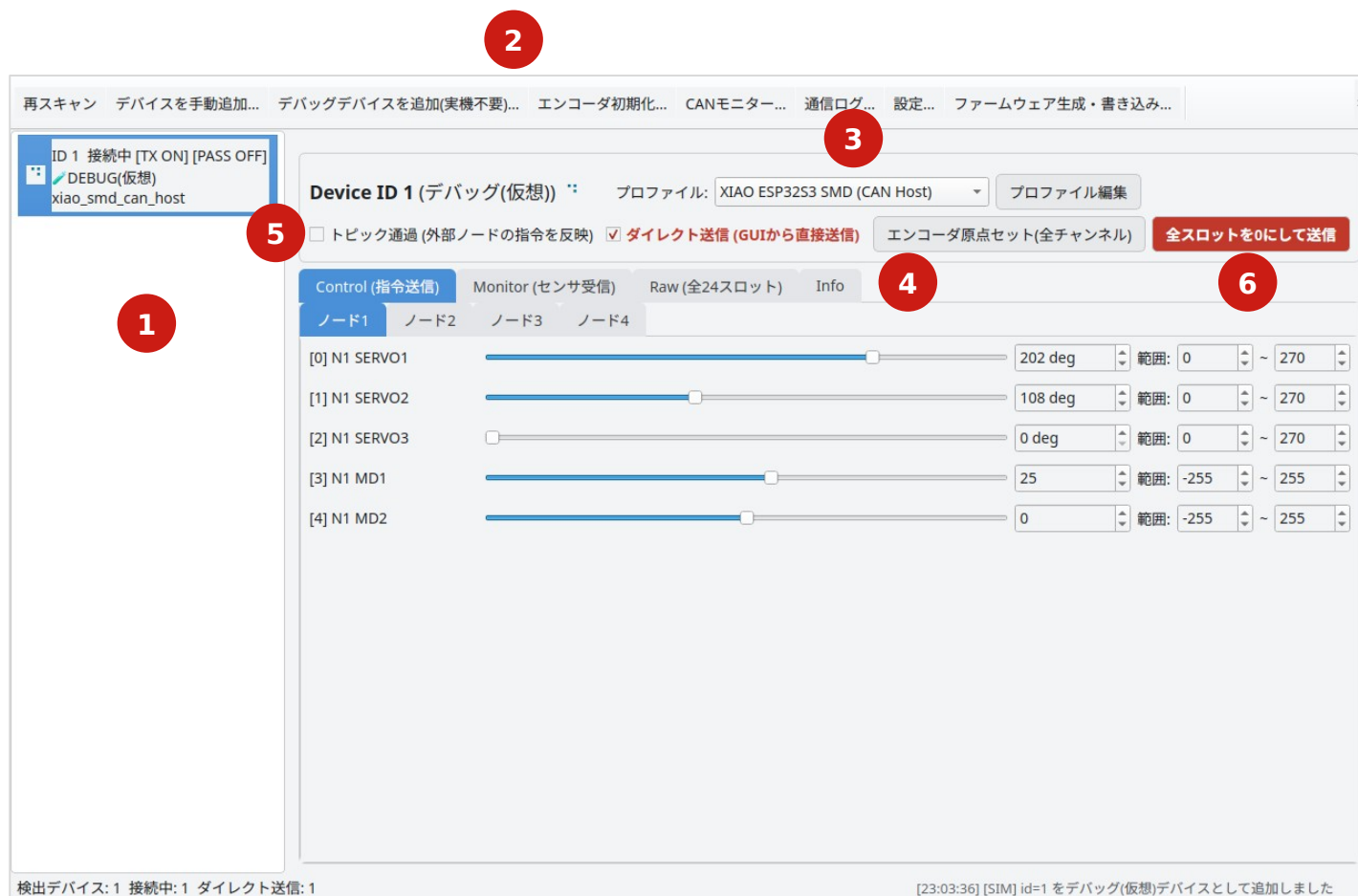
3. 起動

```
ros2 run ros2can ros2can
```

yaml のパラメータを読むなら

```
ros2 launch ros2can ros2can.launch.py
```

画面構成



Control タブ: スライダーで指令値を編集し、ダイレクト送信を ON にした状態

- 1 デバイス一覧
検出したマイコン。右クリックで削除
- 2 ツールバー
再スキャン・FW 生成・E-STOP など
- 3 プロファイル
24 スロットの解釈(ノード構成)を選ぶ
- 4 タブ
Control / Monitor / Raw / Info
- 5 トピック通過 / ダイレクト送信
外部ノードの指令か、GUI の値か
- 6 全スロットを 0 にして送信
そのデバイスへゼロ指令

実機なしで試す: デバッグデバイス

1 「デバッグデバイスを追加(実機不要)…」

DEVICE_ID (例: 1) を入力すると仮想デバイスが一覧に出る

2 Control タブでスライダーを動かす

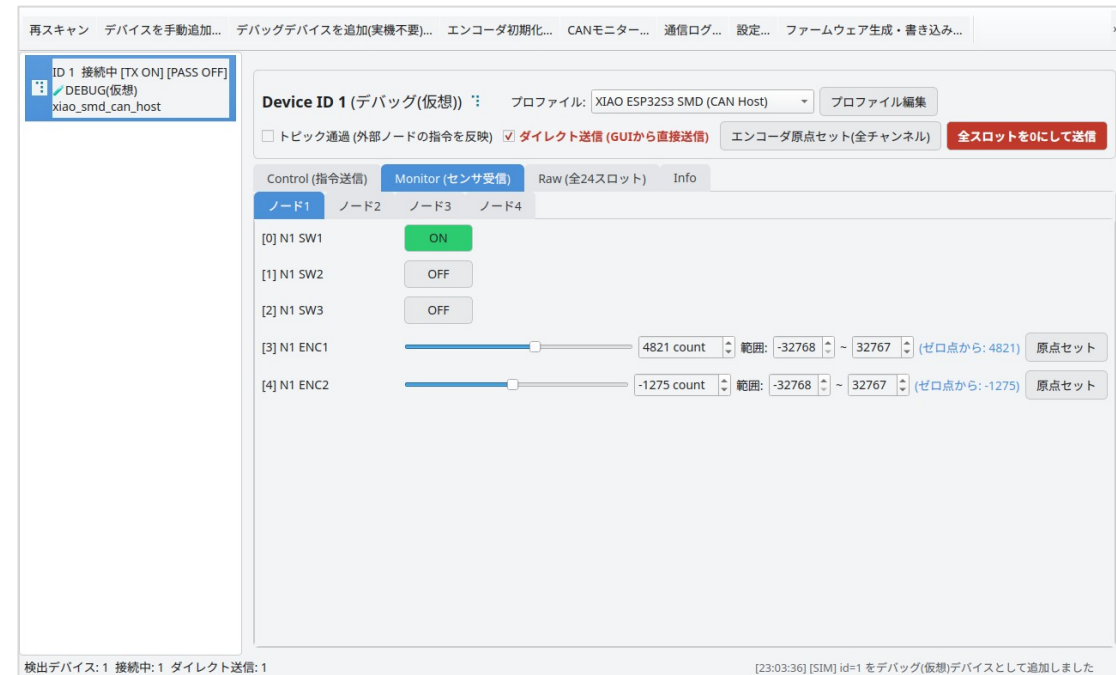
右上の「ダイレクト送信」にチェックを入れると送信開始

3 Monitor / Raw / Info で値を確認

書いた値が仮想デバイスの RX にループバックされる

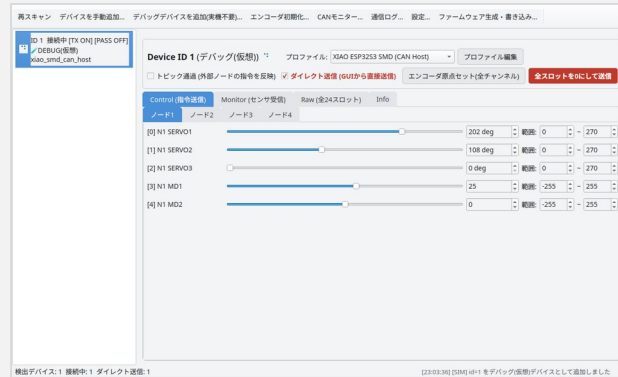
4 スイッチやカウンタを手入力

Monitor タブで実機のセンサ入力を再現できる



仮想デバイスも実機と同じく `serial_rx_[ID]` を Publish / `serial_tx_[ID]` を Subscribe するので、`rqt` や自作ノードのテストにそのまま使えます。

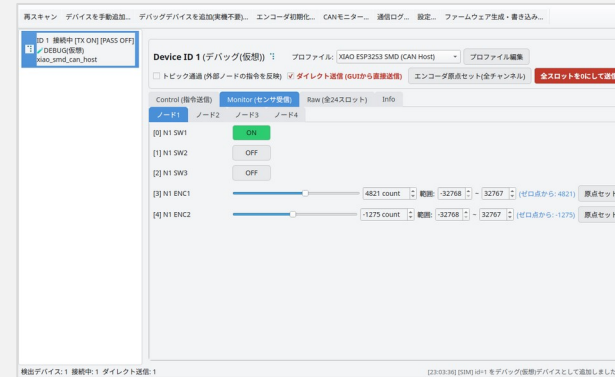
4つのタブ



Control

指令送信

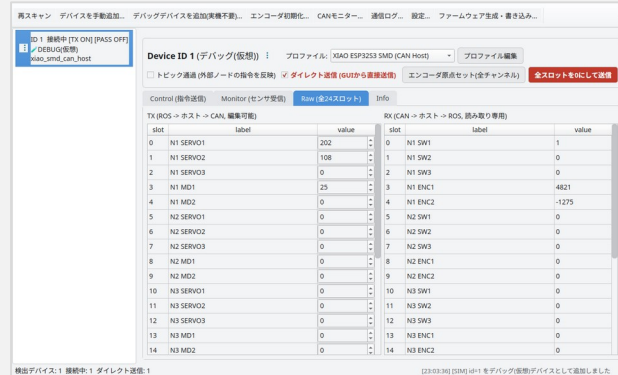
サーボ・モータ等へ指令。
送信には「ダイレクト送信」
ON が必要



Monitor

センサ受信

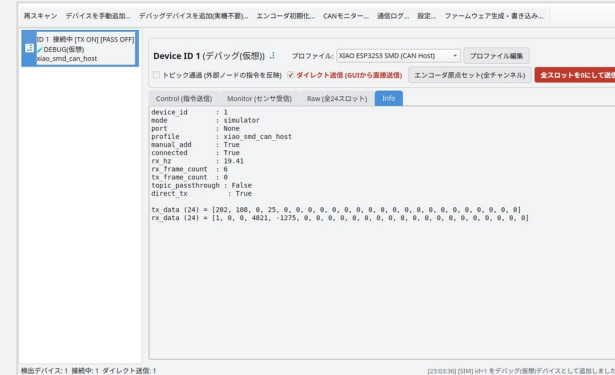
スイッチ・エンコーダ等を取り
アルタイム表示。「原点セッ
ト」も可



Raw

全 24 スロット

プロフィールを介さず生の
24 スロットを直接編集・確
認



Info

通信状態

接続状態・RX 周波数・フ
レーム数。通信トラブルの
切り分けに

安全機能と止め方

1

ダイレクト送信は既定 **OFF**

起動直後は何も送信されない。値を確認してから ON にする

2

■ 全デバイス **E-STOP**

全デバイスにゼロ指令を送り、送信を無効化。迷ったらこれ

3

ウィンドウを閉じる=ゼロ指令

閉じるだけでも安全側に倒れる

4

モード値 **0** で安全停止

CubeMars / ロボマスは全ゼロで速度ループ・目標 0 =その場で停止

注意

ファームウェアに通信途絶時のフェイルセーフはありません。

CAN・シリアルが途切れても、マイコンは最後に受け取った指令を保持し続けます。

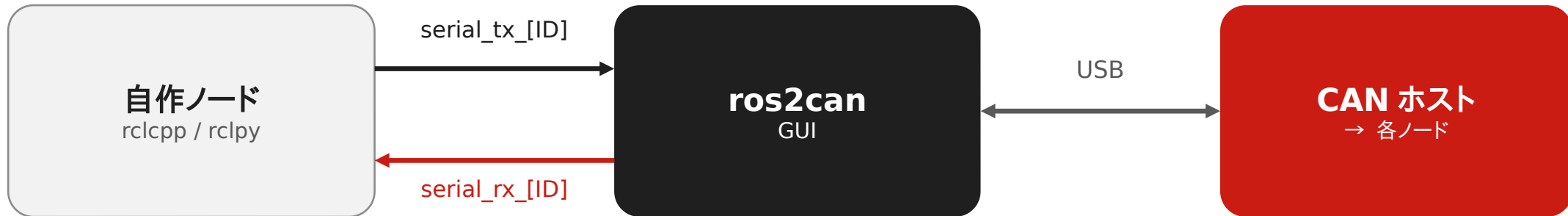
自作ノードで動かす場合は、ノード終了時にゼロ指令を送る処理を必ず入れてください(例: `on_shutdown`)。

03

自作ノードから動かす

トピック・スロット割当・サンプルコード

トピック・インターフェース



トピック	型	向き	内容
<code>serial_tx_[ID]</code>	<code>Int16MultiArray</code>	ROS → ホスト	制御指令 (24 スロット)
<code>serial_rx_[ID]</code>	<code>Int16MultiArray</code>	ホスト → ROS	センサ値 (24 スロット、serial_bridge 互換)
<code>serial_rx_[ID]_unwrapped</code>	<code>Int32MultiArray</code>	ホスト → ROS	エンコードのオーバーフローを展開した積算値
<code>serial_rx_[ID]_zeroed</code>	<code>Int32MultiArray</code>	ホスト → ROS	「原点セット」時点からの相対値
<code>zero_channel_request</code>	<code>Int32MultiArray</code>	ROS → ros2can	[device_id, ch] で原点セット (ch<0 で全 ch)

24 スロットの割当 (既定プロファイル)



$$\text{index} = \text{node} \times 5 + \text{local}$$

node はコード上 0 始まり (ノード 1 → 0)

local	指令 (TX)	帰還 (RX)
0	SERVO1	SW1
1	SERVO2	SW2
2	SERVO3	SW3
3	(予備)	ENC1
4	(予備)	ENC2

プロファイルとは

- 24 スロットをどう解釈するかの設定セット。デバイスごとに切り替えられる
- 組み込み: SMD / MES / SS (CAN Host)、FOC、robomas_driver、cubemars_ak_driver、generic_raw
- ノード数・スロット数を変えたら「プロファイル編集」で作成 (~/.config/ros2can/profiles/ に保存)

サンプル:サーボを動かす

```
#include "common/Ros2CanPacketController.hpp"
Ros2CanPacketController pkt;

// 1) 指令を publish ( ID = ホストの DEVICE_ID )
pub_ = create_publisher<Int16MultiArray>("serial_tx_1", 10);

// 2) 一定周期で送り続ける
void timer_callback() {
    pkt.setServo(/*node*/0, /*servo*/1, 90); // ノード 1 の SERVO1 を 90 度
    Int16MultiArray msg; msg.data = pkt.toVector();
    pub_>publish(msg);
}

// 3) センサ値を受け取る
void sensor_callback(const Int16MultiArray::SharedPtr m) {
    pkt.updateRx(m->data);
    bool sw1 = pkt.getSW(0, 1); int16_t enc1 = pkt.getEnc(0, 1);
}

// 終了時にゼロ指令(ノード側の責務)
rclcpp::on_shutdown([this]{ send_zero_and_stop(); });
```

主なメソッド

メソッド	内容
setServo(n, k, deg)	SERVOk を角度指令
getSW(n, k)	SWk の状態
getEnc(n, k)	ENCK のカウント
updateRx(data)	受信値を反映
toVector()	送信用配列を取得

n はノード番号 (0 始まり、ノード 1 → 0)

完全版: ros2can_example

CubeMars 用:

Ros2CanCubemarsPacketController

CLI で疎通確認・GUI との関係

```
# 受信を眺める
ros2 topic echo /serial_rx_1

# 1 回だけ送る(先頭 =SERVO1 に 90)
ros2 topic pub --once /serial_tx_1 \
  std_msgs/msg/Int16MultiArray \
  "{data: [90,0,0,0,0, 0,0,0,0,0, 0,0,0,0,0,
    0,0,0,0,0, 0,0,0,0]}"

# デバイス 1 の全チャンネルを原点セット
ros2 topic pub --once /zero_channel_request \
  std_msgs/msg/Int32MultiArray "{data: [1, -1]}"
```

--once を忘れずに

ros2 topic pub は既定で周期送信。忘れると指令が送られ続けます

外部ノードの指令が届く条件

トピック通過	ダイレクト送信	実機に届く値
ON (既定)	OFF (既定)	外部ノード
ON / OFF	ON	GUI の値
OFF	OFF	送信されない

- ダイレクト送信 ON のデバイスは serial_tx_[ID] を無視します
- ダイレクト送信中は 20Hz で現在値を周期送信します
- 実機が無くても「デバイスを手動追加…」でトピックを先に確認できます

04

応用

CubeMars AK・DJI ロボマス・ファームウェア生成

CubeMars AK (MODE_CUBEMARS)

独立デバイスとして AK シリーズ最大 4 台を直接制御。アクチュエータ内蔵のクローズドループを使うので、ホスト側で PID は不要です。

control_mode	モード	target の単位
0	速度ループ	10 ERPM / LSB
1	位置ループ	0.1 deg / LSB
2	MIT (Force Control)	0.1 deg / LSB

$$\text{torque} = K_p \cdot (\text{pos_des} - \text{pos}) + K_d \cdot (\text{vel_des} - \text{vel}) + \tau_{ff}$$

- K_p を下げると外力を柔らかく受け流す関節になる
- $K_p=0 + \tau_{ff}$ でトルク制御(重力補償など)にも
- K_p/K_d は小さい値から少しずつ上げる

角度には上限があります

$\pm 716.2^\circ$

MIT/ 位置の出力軸上限 ($\pm 12.5 \text{ rad}$)

- AK シリーズ共通の MIT プロトコルの固定小数点の限界で、設定では変えられない
- 範囲外は無警告でクランプ → 「ある角度から先が動かない」
- 負荷側 = $\pm 716.2^\circ \div \text{減速比 (13.7:1 で約 } \pm 52^\circ \text{)}$
- 連続回転の関節は速度ループを使う

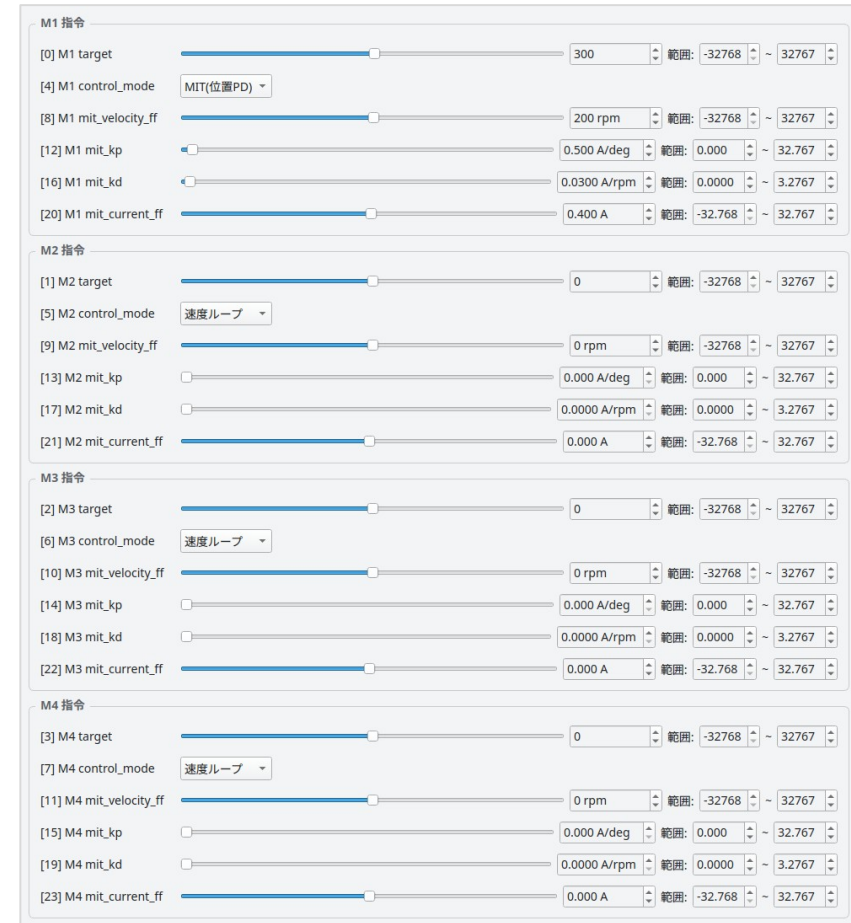
DJI ロボマス (MODE_ROBOMAS)

M3508 / M2006 / GM6020 のいずれか 1 機種を最大 4 台。機種は config.hpp の ROBOMAS_MOTOR_TYPE で固定します。

control_mode	モード	内容
0 (既定)	速度ループ	target = 出力軸 rpm (PID ゲインは FW 固定)
1	MIT (位置 PD)	target = 出力軸角度 1deg/LSB

CubeMars との違い

- ロボマスの ESC は電流指令しか受け付けられないため、位置 PD ループをマイコン (robomas.cpp) で計算
- mit_kp / mit_kd / mit_current_ff は CAN 経由で毎周期変更できる
- 指令送信は 200Hz 固定。CubeMars と同じ 1Mbps なので同一バスに混在できる (CAN ID の重複に注意)



The screenshot displays a control interface for four motors (M1, M2, M3, M4). Each motor has a set of parameters that can be adjusted via sliders and dropdown menus. The parameters include target position/velocity, control mode (e.g., MIT position PD, velocity loop), and feedback gains (mit_kp, mit_kd, mit_current_ff). The interface is organized into sections for each motor, with a consistent layout for parameter adjustment.

ファームウェア生成・書き込み

1

テンプレートを確認

firmware/xiao-esp32-s3_can2io が自動検出される

2

設定を入力

DEVICE_ID・CAN_ID・MODE・BOARD_VARIANT・MULTI・サーボ範囲など

3

「生成」

config.hpp への変更点を確認 → generated_firmware/< 名前 >/ に生成

4

プロジェクトとポートを選んで「書き込み」

pio run -t upload を実行。ポート専有は自動で一時解除

必要なもの: PlatformIO CLI (pip install -U platformio)

生成
xiao-esp32-s3_can2io をテンプレートに、DEVICE_ID・CAN_ID・モード・基板(BOARD_VARIANT)・MULTI1-3・ENC1_MD/ENC2_MD/ENC2_SW・サーボ設定・高度な設定を反映したプロジェクト一式を generated_firmware/<名前>/へ生成します(テンプレート自体は書き換えません)。ROBOMASのPIDゲイン等・その他の設定(チューニング値)には一切触れません。
テンプレート: /home/dev/ros2_ws/src/ros2can/firmware/xiao-esp32-s3_can2io 変更...
生成先の名前: node1_cubemars
生成先: /home/dev/ros2_ws/src/ros2can/generated_firmware/node1_cubemars
DEVICE_ID: 101
CAN_ID (下2桁=ノード番号): 101
MODE: CubeMars AK
BOARD_VARIANT: SOKI (soki本体基板・既存のピン配置)
MULTI1: スイッチ入力 (0)

書き込み
生成済みのプロジェクトを、選択したポートへ pio run -t upload で書き込みます。書き込み中は対象ポートの他の通信は一時的に停止します。
プロジェクト: soki_robomas 更新 削除
ポート: /dev/ttyACM0 再スキャン

書き込み 中断 閉じる

まとめ・参考資料

配線

PC とは USB1 本、残りは CAN でデジチェーン

GUI

実機なしでも仮想デバイスで試せる。止めるときは E-STOP

自作ノード

serial_tx_[ID] / serial_rx_[ID]。Ros2CanPacketController で安全に

応用

CubeMars・ロボマスの専用モード、FW 生成・書き込みも GUI で

参考資料

マニュアル(全機能)

rrst-nhk-project.github.io/ros2can/

クイックスタート

rrst-nhk-project.github.io/ros2can/quickstart/

技術マニュアル(内部実装編)

rrst-nhk-project.github.io/ros2can/internals

GitHub

github.com/RRST-NHK-Project/ros2can

基板(**kicad-ws**)

[RRST-NHK-Project/kicad-ws](#) / XIAO ESP32-S3-SMD